Author: Trevor Robbins
Class: CS431
Professor: Choi
Date: 10/30/26

Analysis of Prefix-Sum Serial vs. Parallel Algorithms (Hillis–Steele and Blelloch Scan)

In this homework assignment, I demonstrate the performance differences between three implementations of the prefix sum. The prefix sum is an operator that, given an array of size n, iterates up to n while calculating a running sum across the elements. For this case study, I use a serial version as the control and compare it against the Hillis–Steele and Blelloch algorithms, with the number of CPU threads as an adjustable parameter set to 2, 4, 8, 16, 32, and 64.

Serial vs. Hillis–Steele vs. Blelloch Scan

The serial implementation is practically identical to the definition above: the iterator runs from 1 to n, carrying the running total forward until the element at n has been summed. This serial algorithm has bottlenecks that parallelization can capitalize on, which is where the Hillis–Steele parallel algorithm comes in. Hillis–Steele iterates over the array while doubling the stride (offset) at each step, so each element's partial sum propagates logarithmically across the array. In turn, this algorithm performs an extra $\log_2(n)$ factor of work per element. With n processors this amortizes to $O(\log n)$ time; however, because we only use up to 64 CPU threads, the cost simplifies to $O(n \log n)$ total work.

Finally, there is the Blelloch scan. This algorithm builds the running sum with an up-sweep (a tree of partial sums) followed by a down-sweep (propagating those sums back down the tree). The tree has a depth of $\log_2(n)$, and the total work is $2(n-1)$ additions, giving $O(n)$ complexity.

Analysis

Table 1 lists the mean runtimes over 20 runs and the resulting speedups for each thread count, and Figure 1 plots them.

Table 1: Mean runtime (seconds) and speedup relative to the serial baseline, averaged over 20 runs per thread count.

Threads	Runs	Serial (s)	Hillis–Steele (s)	Blelloch (s)	Hillis–Steele Speedup	Blelloch Speedup
2	20	0.0240	0.1806	0.0589	0.133	0.407
4	20	0.0239	0.1227	0.0267	0.195	0.897
8	20	0.0249	0.0475	0.0173	0.525	1.440
16	20	0.0250	0.0450	0.0148	0.557	1.688
32	20	0.0250	0.0440	0.0142	0.568	1.765
64	20	0.0249	0.0452	0.0142	0.551	1.751

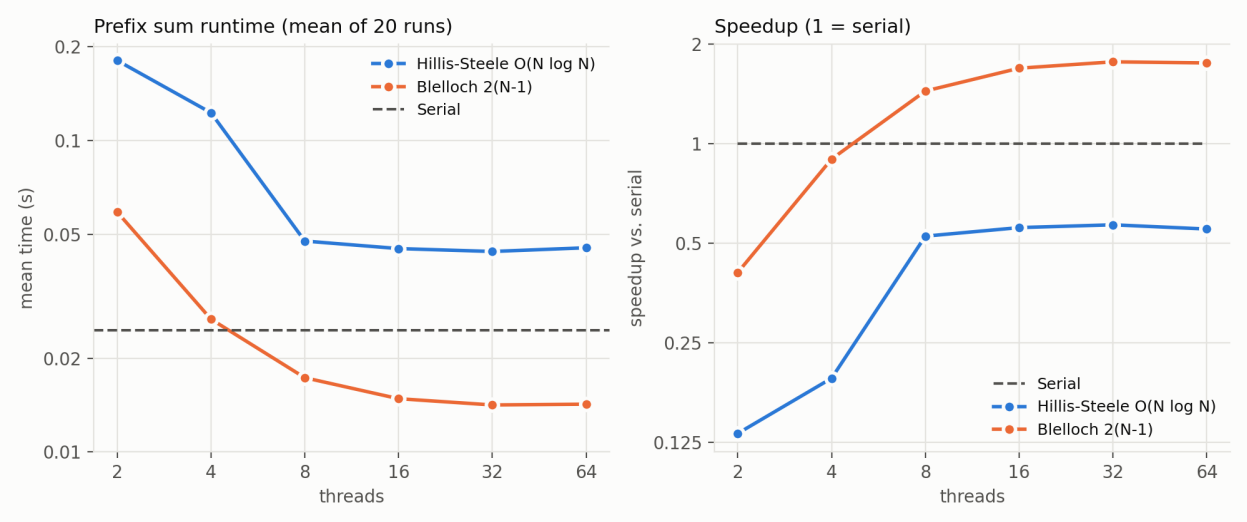


Figure 1: Left: mean prefix-sum runtime versus thread count (log scale). Right: speedup relative to the serial implementation (log scale; 1 = serial).

Reading the results above, the serial implementation stays controlled and fixed against our experimental groups. First, we can see that Hillis–Steele *decreases* performance relative to serial when the thread count is limited to CPU capacities. This is expected: Hillis–Steele is known as a "work-inefficient" algorithm, which makes up for its extra work by spreading it across a hypothetically large number of SIMD-style processing elements. In this case, since we cap at only 64 threads, the extra work is simply multiplied across each core, giving $O(n \log n)$ total work and a performance of only $0.551\times$ the serial baseline at 64 threads.

Analyzing the Blelloch scan, the results stand out as quite interesting. Being referred to as "work efficient," the Blelloch scan performs $2(n-1)$ total additions, and, as mentioned earlier, the tree it builds has a depth of $\log_2(n)$. In our graph, performance increases as the thread count grows and levels off, reaching up to a $1.75\times$ speedup. The apparent logarithmic trend might be a coincidence of simply scaling the thread count by powers of two, but I found it to be an interesting insight into the results as the depth of the work tree is $\log_2(n)$, potentially highlighting a bottleneck for parallelization in this algorithm. Regardless, I noticed that these results appear bottlenecked by memory bandwidth, and further investigation into this algorithm would involve a breakdown of how memory bandwidth affects timing.